Assignment 4:
The Tetris Autoplayer Challenge

Workflow, Game Mechanics, and AI Strategy

Martin Benning

University College London
ENGF0034 – Design and Professional Skills

6 November 2026



Presentation Overview

Learning Objectives

By the end of this briefing, you will be able to:

1. Explain the goals, grading thresholds, and submission rules for Assignment 4.
2. Enrol via UCL GitLab and manage submissions within autograder limits.
3. Run the provided interfaces and apply the scoring rules, bombs, and discards.
4. Inspect `Board` state in `board.py` and implement a custom agent in `player.py`.
5. Combine sandbox simulation, target-state search, heuristics, and Python generators.

Presentation Roadmap

Assignment Overview and Grading
GitLab Workflow and Autograder
Codebase Interfaces and Game Mechanics
Implementing `player.py` and Inspecting `Board`
Search Strategy and Conclusion

Presentation Overview

Learning Objectives

By the end of this briefing, you will be able to:

1. Explain the goals, grading thresholds, and submission rules for Assignment 4.
2. Enrol via UCL GitLab and manage submissions within autograder limits.
3. Run the provided interfaces and apply the scoring rules, bombs, and discards.
4. Inspect `Board` state in `board.py` and implement a custom agent in `player.py`.
5. Combine sandbox simulation, target-state search, heuristics, and Python generators.

Presentation Roadmap

Assignment Overview and Grading
GitLab Workflow and Autograder
Codebase Interfaces and Game Mechanics
Implementing `player.py` and Inspecting `Board`
Search Strategy and Conclusion

Presentation Overview

Learning Objectives

By the end of this briefing, you will be able to:

1. Explain the goals, grading thresholds, and submission rules for Assignment 4.
2. Enrol via UCL GitLab and manage submissions within autograder limits.
3. Run the provided interfaces and apply the scoring rules, bombs, and discards.
4. Inspect `Board` state in `board.py` and implement a custom agent in `player.py`.
5. Combine sandbox simulation, target-state search, heuristics, and Python generators.

Presentation Roadmap

Assignment Overview and Grading
GitLab Workflow and Autograder
Codebase Interfaces and Game Mechanics
Implementing `player.py` and Inspecting `Board`
Search Strategy and Conclusion

Presentation Overview

Learning Objectives

By the end of this briefing, you will be able to:

1. Explain the goals, grading thresholds, and submission rules for Assignment 4.
2. Enrol via UCL GitLab and manage submissions within autograder limits.
3. Run the provided interfaces and apply the scoring rules, bombs, and discards.
4. Inspect `Board` state in `board.py` and implement a custom agent in `player.py`.
5. Combine sandbox simulation, target-state search, heuristics, and Python generators.

Presentation Roadmap

Assignment Overview and Grading
GitLab Workflow and Autograder
Codebase Interfaces and Game Mechanics
Implementing `player.py` and Inspecting `Board`
Search Strategy and Conclusion

Presentation Overview

Learning Objectives

By the end of this briefing, you will be able to:

1. Explain the goals, grading thresholds, and submission rules for Assignment 4.
2. Enrol via UCL GitLab and manage submissions within autograder limits.
3. Run the provided interfaces and apply the scoring rules, bombs, and discards.
4. Inspect `Board` state in `board.py` and implement a custom agent in `player.py`.
5. Combine sandbox simulation, target-state search, heuristics, and Python generators.

Presentation Roadmap

Assignment Overview and Grading
GitLab Workflow and Autograder
Codebase Interfaces and Game Mechanics
Implementing `player.py` and Inspecting `Board`
Search Strategy and Conclusion

Presentation Overview

Learning Objectives

By the end of this briefing, you will be able to:

1. Explain the goals, grading thresholds, and submission rules for Assignment 4.
2. Enrol via UCL GitLab and manage submissions within autograder limits.
3. Run the provided interfaces and apply the scoring rules, bombs, and discards.
4. Inspect `Board` state in `board.py` and implement a custom agent in `player.py`.
5. Combine sandbox simulation, target-state search, heuristics, and Python generators.

Presentation Roadmap

Assignment Overview and Grading
GitLab Workflow and Autograder
Codebase Interfaces and Game Mechanics
Implementing `player.py` and Inspecting `Board`
Search Strategy and Conclusion

Assignment Overview and Grading

Assignment 4: Developing a Tetris Autoplayer

The Core Task

Using the provided Python game engine, implement an autonomous agent in `player.py` that replaces `RandomPlayer` and maximises the game score.

Grading Criteria and Deadline

- **Pass (40%):** Outperform the baseline `RandomPlayer`.
- **Full Marks (100%):** Achieve the highest median score in the cohort.
- **Deadline: Wednesday 25 November 2026, 23:59 UK time**.

Connections to Weeks 3–5

Uses $\mathcal{O}(1)$ `set` lookups in `board.cells`, `yield` generators (Week 4), and sandbox heuristic search (Week 5).

Libraries and Academic Integrity

Use the **Python standard library only** (packages like `numpy` crash on the server). Generative AI tools and external Tetris code are prohibited.

Assignment 4: Developing a Tetris Autoplayer

The Core Task

Using the provided Python game engine, implement an autonomous agent in `player.py` that replaces `RandomPlayer` and maximises the game score.

Connections to Weeks 3–5

Uses $\mathcal{O}(1)$ `set` lookups in `board.cells`, `yield` generators (Week 4), and sandbox heuristic search (Week 5).

Grading Criteria and Deadline

- **Pass (40%):** Outperform the baseline `RandomPlayer`.
- **Full Marks (100%):** Achieve the highest median score in the cohort.
- **Deadline: Wednesday 25 November 2026, 23:59 UK time**.

Libraries and Academic Integrity

Use the **Python standard library only** (packages like `numpy` crash on the server). Generative AI tools and external Tetris code are prohibited.

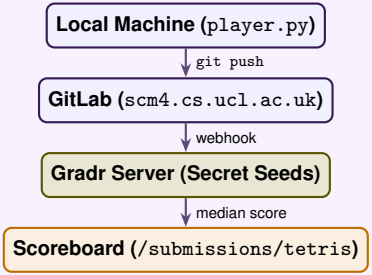
GitLab Workflow and Autograder

Enrolment and Git Development Workflow

Five-Step Setup and Submission

1. **GitLab Account:** Register at `scm4.cs.ucl.ac.uk` with your `@ucl.ac.uk` email. **Please use the identical name that you use on Moodle when registering**.
2. **Enrol on Gradr:** Sign in at `engf0002.cs.ucl.ac.uk` via GitLab, authorise access, and click “Enrol” on Tetris to fork the boilerplate repository.
3. **Clone:** Clone `scm4.cs.ucl.ac.uk/USER/tetris.git` locally (SSH keys optional).
4. **Develop:** Edit `player.py` and test locally.
5. **Submit:** Commit and `git push` to trigger grading.

Submission Pipeline



Autograder Mechanics and Queue Discipline

How Submissions Are Evaluated

1. **Multiple Runs:** Every `git push` runs your player across several independent games.
2. **Secret Random Seeds:** Each run generates a different pseudo-random sequence of tetrominoes.
3. **Median Metric:** Your recorded score is the **median score** across all runs for that commit.
4. **Isolated Server State:** The test harness maintains its own copy of game rules and `board.score`.

Fair Queuing Algorithm

When busy, the server selects next from the user whose most recent evaluated run finished **longest ago**. Rapid repeated pushes place your newest commit at the back of your own queue.

Submission Advice

Expect multi-hour queues near the deadline. Benchmark locally first, push only substantial improvements, and cancel superseded queued submissions via the web interface.

Autograder Mechanics and Queue Discipline

How Submissions Are Evaluated

1. **Multiple Runs:** Every `git push` runs your player across several independent games.
2. **Secret Random Seeds:** Each run generates a different pseudo-random sequence of tetrominoes.
3. **Median Metric:** Your recorded score is the **median score** across all runs for that commit.
4. **Isolated Server State:** The test harness maintains its own copy of game rules and `board.score`.

Fair Queuing Algorithm

When busy, the server selects next from the user whose most recent evaluated run finished **longest ago**. Rapid repeated pushes place your newest commit at the back of your own queue.

Submission Advice

Expect multi-hour queues near the deadline. Benchmark locally first, push only substantial improvements, and cancel superseded queued submissions via the web interface.

Autograder Container Limits

Hard Resource and Execution Bounds

Constraint	Outcome When Reached
256 MB RAM	Process terminated; **not counted**
Runtime exception	Process crashes; **not counted**
50% of 1 CPU core	Caps compute throughput
No disk / network	Read-only sandboxed container
400 falling blocks	Game stops cleanly; **score kept**
15 min wall time	Game stops cleanly; **score kept**

Python Runtime

The grading container runs vanilla **Python 3.7**. Avoid language syntax introduced only in newer Python releases (such as `match`/`case`) and avoid all third-party imports.

Strategic Consequence

Because play stops at 400 blocks, pure survival yields a low score. Your algorithm must maximise points per block while staying fast and memory-light.

Autograder Container Limits

Hard Resource and Execution Bounds

Constraint	Outcome When Reached
256 MB RAM	Process terminated; **not counted**
Runtime exception	Process crashes; **not counted**
50% of 1 CPU core	Caps compute throughput
No disk / network	Read-only sandboxed container
400 falling blocks	Game stops cleanly; **score kept**
15 min wall time	Game stops cleanly; **score kept**

Python Runtime

The grading container runs vanilla **Python 3.7**. Avoid language syntax introduced only in newer Python releases (such as `match`/`case`) and avoid all third-party imports.

Strategic Consequence

Because play stops at 400 blocks, pure survival yields a low score. Your algorithm must maximise points per block while staying fast and memory-light.

Codebase Interfaces and Game Mechanics

Running the Game: Interfaces and Manual Controls

Provided Front-End Views

- **Tkinter (`python3 visual.py`):** Default GUI; convenient for visual debugging, slower when autoplaying.
- **Pygame (`python3 visual-pygame.py`):** Fastest graphical view (`pip install --user pygame`); **recommended for local testing**.
- **Terminal (`python3 cmdline.py`):** Text-only UI (`windows-curses` needed on Windows).

Manual Play (`-m` Flag)

Run `python3 visual.py -m` to play manually at constant fall speed.

Manual Keybindings

Key	Action
← / → / ↓	Move left, right, or down
↑ or `x`	Rotate clockwise
`z`	Rotate anticlockwise
`Space`	Drop block immediately
`d`	Discard current block (10 max)
`b`	Switch next block to bomb (5 max)
`Esc`	Exit game

Game Mechanics: Non-Linear Scoring

Line-Clear Points Table

Clearing multiple rows at once multiplies rewards by $4\times$ per extra row (plus 1 point per row dropped):

Rows Cleared Simultaneously	Points
1 row (Single)	25
2 rows (Double)	100
3 rows (Triple)	400
4 rows (“Tetris”)	**1,600**

Incentive for Multi-Row Clears

A single 4-row “Tetris” clear (1,600 points) equals **64 single-row clears**. With only 400 blocks available, clearing single lines one by one limits your ceiling dramatically.

Implementation Caveat

In `board.py`, `scores = [0, 25, 100, 400, 1600]` only indexes up to 4 rows at once. Never design setups that clear more than 4 rows in a single placement.

Game Mechanics: Non-Linear Scoring

Line-Clear Points Table

Clearing multiple rows at once multiplies rewards by $4\times$ per extra row (plus 1 point per row dropped):

Rows Cleared Simultaneously	Points
1 row (Single)	25
2 rows (Double)	100
3 rows (Triple)	400
4 rows (“Tetris”)	**1,600**

Incentive for Multi-Row Clears

A single 4-row “Tetris” clear (1,600 points) equals **64 single-row clears**. With only 400 blocks available, clearing single lines one by one limits your ceiling dramatically.

Implementation Caveat

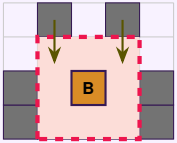
In `board.py`, `scores = [0, 25, 100, 400, 1600]` only indexes up to 4 rows at once. Never design setups that clear more than 4 rows in a single placement.

Modified Mechanics: Discards and Bombs

Two Special Actions

- **Discard (`Action.Discard`, 10 per game):** Removes the currently falling tetromino and spawns `board.next` immediately.
- **Bomb (`Action.Bomb`, 5 per game):** Substitutes a 1×1 bomb (`Shape.B`) in place of `board.next`. When landed at (b_x, b_y), it destroys all occupied cells within $|x - b_x| \leqslant 1, |y - b_y| \leqslant 1$ for zero points, and drops any overhanging blocks in those three columns straight down.

Bomb 3×3 Blast Neighbourhood



Recommended Progression

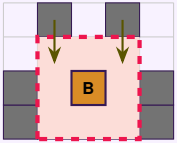
Ignore Bombs and Discards initially. Build a strong core Tetris player first; add special actions only as a later refinement.

Modified Mechanics: Discards and Bombs

Two Special Actions

- **Discard (`Action.Discard`, 10 per game):** Removes the currently falling tetromino and spawns `board.next` immediately.
- **Bomb (`Action.Bomb`, 5 per game):** Substitutes a 1×1 bomb (`Shape.B`) in place of `board.next`. When landed at (b_x, b_y), it destroys all occupied cells within $|x - b_x| \leqslant 1, |y - b_y| \leqslant 1$ for zero points, and drops any overhanging blocks in those three columns straight down.

Bomb 3×3 Blast Neighbourhood



Recommended Progression

Ignore Bombs and Discards initially. Build a strong core Tetris player first; add special actions only as a later refinement.

Implementing `player.py` and Inspecting `Board`

Structuring Your Agent in `player.py`

```python
from board import Direction, Rotation, Action

class Player:
  def choose_action(self, board):
    raise NotImplementedError

class MyTetrisAI(Player):
  def __init__(self, seed=None):
    # Initialise heuristic weights
    pass

  def choose_action(self, board):
    # Return single move, list, or generator
    return Direction.Drop

SelectedPlayer = MyTetrisAI
```

Class Architecture

Subclass `Player` in `player.py` and implement `choose_action(self, board)`.

Two Essential Checks

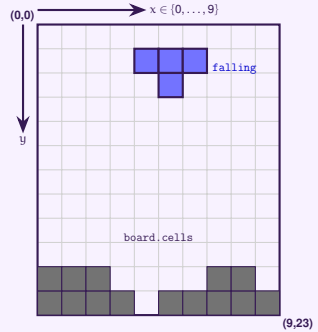
1. Set `SelectedPlayer = MyTetrisAI` at the bottom (exact `CamelCase`).
2. Omit `time.sleep(0.5)` and `print_board` from `RandomPlayer`.

Inspecting Game State: `board.py` Attributes

Key Properties on `board`

- `board.width` (10) and `board.height` (24).
- `board.cells`: Python `set` of `(x, y)` tuples for landed blocks ($\mathcal{O}(1)$ lookup; unordered).
- `board.falling`: Active `Block` with `.shape`, `.cells`, and bounds `.left`, `.right`, `.top`, `.bottom`.
- `board.next`: Upcoming `Block` (becomes `None` inside a sandbox after the first block lands).
- `board.score`, `board.discards_remaining`, and `board.bombs_remaining`.

Grid Coordinate System



Simulating Moves in a Cloned Sandbox

Example Sandbox Trial

```
sandbox = board.clone()

landed = sandbox.rotate(Rotation.Anticlockwise)
if not landed:
  landed = sandbox.move(Direction.Left)
if not landed:
  sandbox.move(Direction.Drop)

# Evaluate post-landing sandbox.cells
# and score gain (sandbox.score - board.score)
```

Sandbox Methods on `Board`

`board.clone()` isolates trials (Week 5):

- `move()`: `Left`, `Right`, `Down`, `Drop`
- `rotate()`: `Clockwise`, `Anticlockwise`
- `skip()`, `discard()`, `bomb()`

Implicit Downward Step

Each call also applies the 1-row fall, returning `True` if the block landed.

Returning Actions: Lists vs Python Generators

Option 1: Single Move or List

Return one action (`Direction.*`, `Rotation.*`, `Action.*`, or `None` to skip), or a non-empty list of actions:

```
return [
  Rotation.Clockwise,
  Direction.Left,
  Direction.Drop,
]
```

Option 2: Generator (`yield`, Week 4)

Stream actions lazily one by one without building an explicit list first:

```
yield Rotation.Clockwise
yield Direction.Left
yield Direction.Drop
```

Engine Execution Rule

Actions run only until the falling block lands; leftover actions are ignored and `choose_action` is called for the new block. Never return `[]` (infinite loop).

Returning Actions: Lists vs Python Generators

Option 1: Single Move or List

Return one action (`Direction.*`, `Rotation.*`, `Action.*`, or `None` to skip), or a non-empty list of actions:

```
return [
  Rotation.Clockwise,
  Direction.Left,
  Direction.Drop,
]
```

Option 2: Generator (`yield`, Week 4)

Stream actions lazily one by one without building an explicit list first:

```
yield Rotation.Clockwise
yield Direction.Left
yield Direction.Drop
```

Engine Execution Rule

Actions run only until the falling block lands; leftover actions are ignored and `choose_action` is called for the new block. Never return `[]` (infinite loop).



Search Strategy and Conclusion

Search and Heuristic Strategy Overview

Target-State Search

Because future blocks beyond `board.next` are unknown, search over candidate **landing placements**:

1. Test all rotations (0–3) and columns (0–9) — at most ≈ 40 placements.
2. Simulate each on `board.clone()` ending with `Direction.Drop`.
3. Score the resulting board and pick the maximum.

Lookahead: Extending to `board.next` tests $\approx 1{,}600$ states (respecting $\mathcal{O}(b^d)$ runtime bounds).

Linear Heuristic Evaluation

Combine structural features with tuned weights,

$$E(s) = w_1 \Delta S - w_2 H - w_3 K - w_4 B,$$

where:

- ΔS: Score gain (rewards 3–4 row clears).
- H: Aggregate or maximum column height.
- K: Buried empty holes under blocks.
- B: Surface bumpiness $\sum_x |h_x - h_{x+1}|$.

Summary and Recommended Workflow

Step-by-Step Checklist

1. Register on GitLab, enrol on Gradr, and clone `tetris`.
2. Study `board.py` and `player.py` thoroughly before designing heuristics.
3. Implement your `Player` subclass and set `SelectedPlayer`.
4. Benchmark locally in `visual-pygame.py`.
5. Push substantial improvements early to avoid deadline queues.

Key Reminders

- **Deadline:** Wednesday 25 November 2026, 23:59 UK time.
- **Scoring:** Prioritise multi-row clears (1,600 pts for 4 rows).
- **Limits:** 256 MB RAM, 400 blocks, 15 min, Python standard library only.
- **Progression:** Master core Tetris before adding Bombs or Discards.

Understand the board state first, then refine your heuristics.